

GraphNet Analyzer

Documentação Técnica Detalhada
Análise Arquitetural e de Dependências

Jafte Carneiro Fagundes da Silva
Nicolas Hrescak

Orientador: Prof. Fabrício Enembreck
Pontifícia Universidade Católica do Paraná (PUCPR)
Bacharelado em Ciência da Computação

16 de junho de 2026

Conteúdo

1	Introdução	5
1.1	Objetivos do Projeto	5
1.2	Escopo Técnico	5
2	Arquitetura Geral do Sistema	6
2.1	Visão de Módulos	6
2.2	Diagrama de Dependências entre Pacotes	6
3	Camada de Estrutura de Dados	7
3.1	Classe <code>Graph</code>	7
3.1.1	Responsabilidades	7
3.1.2	Atributos Principais	7
3.1.3	Métodos Públicos	7
3.1.4	Fluxo de Criação	7
4	Camada de Algoritmos	8
4.1	Classe <code>GraphAlgorithms</code>	8
4.1.1	Responsabilidades	8
4.1.2	Algoritmo 1: Conectividade	8
4.1.3	Algoritmo 2: Componentes Conexos	8
4.1.4	Algoritmo 3: Euleriano	9
4.1.5	Algoritmo 4: Detecção de Ciclo	9
4.1.6	Algoritmo 5: Centralidade de Proximidade	9
4.1.7	Algoritmo 6: Centralidade de Intermediação (Brandes)	10
4.1.8	Algoritmo 7: Dijkstra (Auxiliar)	10
4.1.9	Métodos Auxiliares Privados	11
5	Camada de Domínio	12
5.1	Classe <code>SocialNetwork</code>	12
5.1.1	Responsabilidades	12
5.1.2	Fluxo do Método <code>recommendFollower</code>	12
5.1.3	Algoritmo DFS com Rastreamento de Caminho	12
5.1.4	Exemplo de Uso	13
6	Camada de I/O e Utilitários	14
6.1	Classe <code>GraphGenerator</code>	14
6.1.1	Métodos Estáticos	14
6.1.2	Geração de Rede Social	14
6.2	Classe <code>NamesLoader</code>	14
6.3	Classe <code>PajekIO</code>	14
6.3.1	Formato Pajek	14
6.3.2	Métodos Estáticos	14
7	Camada de Apresentação	15
7.1	Classe <code>GraphGUI</code>	15
7.1.1	Responsabilidades	15
7.1.2	Arquitetura Refatorada	15

7.1.3	Componentes da GUI	15
7.2	Classe <code>GraphLoadingHandler</code>	16
7.2.1	Responsabilidades	16
7.2.2	Fluxo de Carregamento	16
7.3	Classe <code>AlgorithmHandler</code>	16
7.3.1	Responsabilidades	16
7.3.2	Métodos Principais	16
7.4	Classe <code>SocialNetworkHandler</code>	16
7.4.1	Responsabilidades	17
7.5	Classe <code>GraphPanel</code>	17
7.5.1	Responsabilidades	17
7.5.2	Algoritmo de Renderização	17
7.6	Classe <code>GraphUIState</code>	17
7.6.1	Estado Gerenciado	18
7.7	Classe <code>Theme</code>	18
7.7.1	Tokens Fornecidos	18
7.8	Componentes Customizados	18
8	Fluxos de Dados Principais	19
8.1	Fluxo 1: Inicialização da Aplicação	19
8.2	Fluxo 2: Geração de Rede Social	19
8.3	Fluxo 3: Execução de Algoritmo (Exemplo: Conectividade)	19
8.4	Fluxo 4: Sistema de Recomendação de Seguidores	20
8.5	Fluxo 5: Centralidade de Intermediação (Brandes)	20
9	Padrões de Design	21
9.1	MVC (Model-View-Controller)	21
9.2	Facade	21
9.3	Dependency Injection	21
9.4	Factory	21
9.5	Strategy	22
9.6	Observer	22
10	Análise de Dependências	23
10.1	Matriz de Dependências entre Classes	23
10.2	Direção das Dependências	23
10.3	Garantias de Acoplamento Baixo	23
11	Análise de Complexidade	24
11.1	Algoritmos	24
11.2	Complexidade para Rede Social (V=5000, E=30000)	24
12	Estrutura de Pacotes e Responsabilidades	25
12.1	Pacote <code>graph.model</code>	25
12.2	Pacote <code>graph.algorithm</code>	25
12.3	Pacote <code>graph.domain</code>	25
12.4	Pacote <code>graph.io</code>	25
12.5	Pacote <code>graph.generator</code>	25
12.6	Pacote <code>graph.gui</code>	25

12.7 Pacote graph (raiz)	25
13 Princípios SOLID Aplicados	26
13.1 S — Single Responsibility Principle	26
13.2 O — Open/Closed Principle	26
13.3 L — Liskov Substitution Principle	26
13.4 I — Interface Segregation Principle	26
13.5 D — Dependency Inversion Principle	27
14 Refatorizações Realizadas	28
14.1 Refatoração 1: Extração de Algoritmos	28
14.2 Refatoração 2: Refactoring de GUI	28
14.3 Refatoração 3: Encapsulamento	28
14.4 Refatoração 4: Eliminação de Duplicação	28
15 Conclusão	29
15.1 Arquitetura Alcançada	29
15.2 Características Principais	29
15.3 Autoria	29
15.4 Recomendações para Expansão Futura	30
A Referências de Código	30
A.1 Arquivo de Nomes	30
A.2 Formato Pajek	30
A.3 Comando de Compilação	30
A.4 Comando de Execução	31

1 Introdução

GraphNet Analyzer é uma aplicação completa de análise e visualização de grafos de alta dimensionalidade, desenvolvida em Java puro (JDK 8+), combinando uma interface gráfica moderna com um sistema de linha de comando legado. O projeto implementa uma rede social com 5.000 usuários e mais de 25.000 conexões direcionadas.

1.1 Objetivos do Projeto

1. Implementar uma estrutura de dados de grafo eficiente com suporte a vértices nomeados e arestas ponderadas
2. Implementar 7 algoritmos clássicos de análise de grafos (conectividade, componentes, ciclos, Euleriano, centralidades)
3. Fornecer uma interface gráfica interativa com visualização em tempo real
4. Implementar um sistema de recomendação de seguidores via busca em profundidade (DFS)
5. Aplicar princípios SOLID para obter código modular, testável e extensível

1.2 Escopo Técnico

- **Linguagem:** Java 8+ (sem dependências externas)
- **GUI:** Swing com componentes customizados
- **Tamanho:** 5.000 vértices, 25.000 a 40.000 arestas
- **Formato de I/O:** Pajek (.net)
- **Padrões de Design:** MVC, Facade, Factory, Strategy, Dependency Injection

2 Arquitetura Geral do Sistema

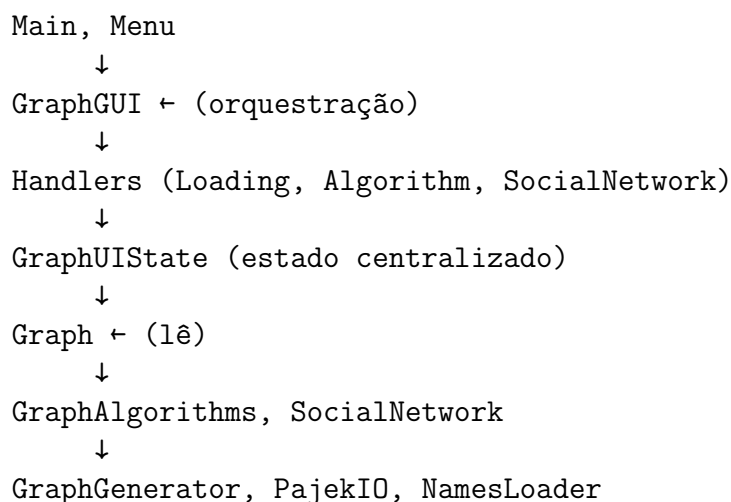
2.1 Visão de Módulos

A aplicação está organizada em 6 camadas funcionais, cada uma com responsabilidade bem definida:

Camada	Pacote	Responsabilidade
Apresentação	<code>graph.gui</code>	Interface gráfica e componentes customizados
Controladores	<code>graph.gui.*Handler</code>	Roteamento de eventos e orquestração
Domínio	<code>graph.domain</code>	Lógica de negócio (recomendações)
Algoritmos	<code>graph.algorithm</code>	Análise de grafos (7 algoritmos)
Estrutura de Dados	<code>graph.model</code>	Representação interna do grafo
I/O e Utilitários	<code>graph.io</code> , <code>graph.generator</code>	Importação, exportação e geração

2.2 Diagrama de Dependências entre Pacotes

As dependências entre pacotes seguem a regra de acoplamento mínimo:



A estrutura garante que:

- **Algoritmos** nunca dependem de GUI
- **Domínio** nunca conhece Handlers
- **Graph** é imutável para leitura segura
- **Handlers** recebem dependências via injeção de construtor

3 Camada de Estrutura de Dados

3.1 Classe Graph

A classe `Graph` é a estrutura de dados central, representando um grafo ponderado, rotulado e opcionalmente direcionado.

3.1.1 Responsabilidades

- Armazenar vértices em um array de strings (`names[]`)
- Armazenar arestas em lista de adjacências (`adj[]`)
- Fornecer API pública para consulta segura de vizinhos
- Gerenciar automaticamente inversas de arestas para grafos não-direcionados

3.1.2 Atributos Principais

Atributo	Tipo	Descrição
<code>numVertices</code>	<code>final int</code>	Número total de vértices
<code>directed</code>	<code>boolean</code>	Se o grafo é direcionado
<code>names[]</code>	<code>private String[]</code>	Rótulos dos vértices
<code>adj[]</code>	<code>private List<int[]>[]</code>	Lista de adjacências (destino, peso)

3.1.3 Métodos Públicos

Método	Descrição e Complexidade
<code>Graph(int, boolean)</code>	Construtor. $O(V)$
<code>setName(int, String)</code>	Define rótulo do vértice. $O(1)$
<code>getName(int)</code>	Retorna rótulo. $O(1)$
<code>addEdge(int, int, int)</code>	Adiciona aresta. $O(1)$ amortizado
<code>findByName(String)</code>	Busca vértice por nome (case-insensitive). $O(V)$
<code>neighbors(int)</code>	Retorna lista de vizinhos. $O(1)$

3.1.4 Fluxo de Criação

Listing 1: Uso da classe `Graph`

```
1 // Cria o
2 Graph graph = new Graph(5000, true); // direcionado
3
4 // Configura o de nomes
5 for (int i = 0; i < 5000; i++) {
6     graph.setName(i, nomes[i]);
7 }
8
9 // Adi o de arestas
10 graph.addEdge(0, 1, 1); // pessoa 0 segue pessoa 1
```

4 Camada de Algoritmos

4.1 Classe GraphAlgorithms

Implementa 7 algoritmos clássicos de análise de grafos, especializados para grafos direcionados com componentes fracamente conectados.

4.1.1 Responsabilidades

- Verificar conectividade (fraca para direcionados)
- Identificar componentes conexos
- Detectar ciclos (coloração DFS)
- Verificar propriedades Eulerianas
- Calcular centralidade de proximidade (Dijkstra)
- Calcular centralidade de intermediação (Algoritmo de Brandes)

4.1.2 Algoritmo 1: Conectividade

Função: `isConnected()`

Entrada: Grafo `G`

Saída: `boolean`

1. Se $|V| = 1$, retorna `true`
2. Cria vetor `visitados[|V|] ← false`
3. Inicia BFS a partir do vértice 0 no grafo não-direcionado
4. Retorna `true` se todos os vértices foram visitados

Complexidade: $O(V + E)$

Caso de Uso: Verificar se a rede social é conexa (fracamente)

4.1.3 Algoritmo 2: Componentes Conexos

Função: `getComponents()`

Entrada: Grafo `G`

Saída: `List<List<Integer>>`

1. Cria vetor `visitados[|V|] ← false`
2. Para cada vértice `i` não visitado:
 - a. Cria nova lista de componentes
 - b. Inicia BFS a partir de `i`
 - c. Marca todos os vértices alcançáveis como visitados
 - d. Adiciona lista de componentes ao resultado
3. Retorna lista de componentes

Complexidade: $O(V + E)$

Caso de Uso: Identificar subgrupos desconectados na rede

4.1.4 Algoritmo 3: Euleriano

Função: `isEulerian()`

Entrada: Grafo G

Saída: 2 (Ciclo) | 1 (Caminho) | 0 (Não Euleriano)

Para grafos NÃO-direcionados:

1. Conta vértices com grau ímpar
2. Se 0 → Ciclo Euleriano (retorna 2)
3. Se 2 → Caminho Euleriano (retorna 1)
4. Senão → Não Euleriano (retorna 0)

Para grafos DIRECIONADOS:

1. Conta out-degree e in-degree de cada vértice
2. Se todos iguais → Ciclo Euleriano (retorna 2)
3. Se exatamente um com out-degree = in-degree + 1
e outro com in-degree = out-degree + 1 → Caminho Euleriano (retorna 1)
4. Senão → Não Euleriano (retorna 0)

Complexidade: $O(V + E)$

Caso de Uso: Verificar se existe caminho/ciclo que visite toda aresta exatamente uma vez

4.1.5 Algoritmo 4: Detecção de Ciclo

Função: `hasCycle()`

Entrada: Grafo G

Saída: boolean

Para grafos DIRECIONADOS (coloração DFS):

1. Cores: BRANCO (0, não visitado), CINZA (1, visitando), PRETO (2, visitado)
2. Para cada vértice não visitado:
 - a. Inicia DFS com cor CINZA
 - b. Se encontrar vizinho CINZA → ciclo detectado, retorna true
 - c. Marca como PRETO ao terminar
3. Retorna false se nenhum ciclo encontrado

Para grafos NÃO-direcionados (rastreamento de pai):

1. Para cada vértice não visitado:
 - a. Inicia DFS
 - b. Se encontrar vizinho visitado (não é pai) → ciclo, retorna true
3. Retorna false se nenhum ciclo encontrado

Complexidade: $O(V + E)$

Caso de Uso: Detectar influências circulares na rede social

4.1.6 Algoritmo 5: Centralidade de Proximidade

Função: `closenessCentrality()`

Entrada: Grafo G ponderado

Saída: double[] closeness

Para cada vértice s :

1. Executa Dijkstra a partir de s
2. Calcula soma de distâncias para todos os outros vértices
3. $\text{closeness}[s] = (|V| - 1) / \text{soma_distâncias}$

Retorna array closeness

Complexidade: $O(V \times (V + E) \log V)$ com Dijkstra e heap binário

Caso de Uso: Identificar usuários centrais na rede (fácil acesso a outros)

4.1.7 Algoritmo 6: Centralidade de Intermediação (Brandes)

Função: `betweennessCentrality()`

Entrada: Grafo G ponderado

Saída: double[] betweenness

Para cada vértice s :

1. Executa Dijkstra a partir de s
2. Constrói lista de predecessores para cada vértice
3. Retrocede do vértice com maior distância até s
4. Acumula dependências: $(s,t) = |\text{predecessores}(t)| \times (s, \text{predecessores}(t)) / |\text{predecessores}(t)|$
5. Adiciona (s,t) à `betweenness[t]`

Divide betweenness por $(V-1) \times (V-2) / 2$ para grafos não-direcionados

Retorna array betweenness

Complexidade: $O(V \times (V + E) \log V)$

Caso de Uso: Identificar usuários que atuam como "ponte" entre grupos

4.1.8 Algoritmo 7: Dijkstra (Auxiliar)

Função: `dijkstra(int source)`

Entrada: Vértice de origem

Saída: int[] distâncias

1. Inicializa distâncias com INFINITO, exceto `source = 0`
2. Cria PriorityQueue com (distância, vértice)
3. Enquanto queue não vazia:
 - a. Extraí (d, u) com menor distância
 - b. Se $d > \text{distância}[u]$, continua (já processado)
 - c. Para cada vizinho v de u :
 - Se $\text{distância}[u] + \text{peso}(u,v) < \text{distância}[v]$:
 - Atualiza $\text{distância}[v]$
 - Adiciona $(\text{nova_distância}, v)$ à queue
4. Retorna array de distâncias

Complexidade: $O((V + E) \log V)$ com heap binário

Caso de Uso: Encontrar caminhos mínimos em redes ponderadas

4.1.9 Métodos Auxiliares Privados

Método	Descrição
bfsUndirected(boolean[])	BFS ignorando direção das arestas
undirectedNeighbors(int)	Retorna vizinhos em ambas direções
dfsColor(int, int[], int)	DFS com coloração (branco/cinza/preto)
dfsCycle(int, int, boolean[], boolean[])	DFS para detectar ciclos (não-direcionado)
dijkstra(int)	Dijkstra para uma fonte
backtrackBrandes(...)	Retrocesso em Brandes

5 Camada de Domínio

5.1 Classe SocialNetwork

Implementa a lógica de negócio específica da rede social — o sistema de recomendação de seguidores.

5.1.1 Responsabilidades

- Validar nomes de usuários no grafo
- Verificar relacionamentos diretos (já segue)
- Encontrar caminhos indiretos via DFS
- Gerar recomendações com caminho completo

5.1.2 Fluxo do Método recommendFollower

Função: `recommendFollower(sourceName, targetName)`

Entrada: Nomes de dois usuários

Saída: String (recomendação com caminho ou motivo da recusa)

1. Valida nomes: busca índices no grafo
 - Se qualquer um não encontrado → "Person(s) not found"
2. Rejeita auto-recomendação: `u == v`
 - Se igual → "Cannot recommend a person to themselves"
3. Verifica relacionamento direto: `(u,v)` já existe?
 - Se sim → "already follows"
4. Busca caminho indireto via DFS
 - Chama `dfsPath(u, v, visited[], path[])`
 - Se caminho encontrado:
 - Constrói string "Recommended! Path found:\n u → ... → v"
 - Senão:
 - "Not recommended. No path exists..."
5. Retorna mensagem com resultado

5.1.3 Algoritmo DFS com Rastreamento de Caminho

Função PRIVADA: `dfsPath(current, dest, visited[], path[])`

Entrada: Vértice atual, destino, visitados, caminho acumulado

Saída: boolean (verdadeiro se destino alcançado)

1. Marca `current` como visitado
2. Adiciona `current` ao final de `path[]`
3. Se `current == dest`, retorna `true` (encontrado!)
4. Para cada vizinho `v` de `current` não visitado:

- a. Recursivamente chama `dfsPath(v, dest, ...)`
- b. Se retornar true, retorna true (propaga sucesso)
5. Remove current do final de `path[]` (backtrack)
6. Retorna false (destino não alcançável daqui)

Complexidade: $O(V + E)$ no pior caso

Propriedade: Encontra o primeiro caminho (não necessariamente o mais curto)

5.1.4 Exemplo de Uso

Listing 2: Sistema de Recomendação de Seguidores

```
1 SocialNetwork network = new SocialNetwork(graph);
2
3 String result = network.recommendFollower("Alice", "Carol");
4 // Possíveis saídas:
5 // 1. "Recommended! Path found: Alice -> Bob -> Carol"
6 // 2. "'Alice' already follows 'Carol'."
7 // 3. "Not recommended. No path exists..."
8 // 4. "Person(s) not found in the graph."
```

6 Camada de I/O e Utilitários

6.1 Classe GraphGenerator

Responsável pela criação de grafos segundo diferentes estratégias.

6.1.1 Métodos Estáticos

Método	Descrição
<code>generateSocialNetwork(String namesFile)</code>	Gera rede social com 5.000 pessoas
<code>generateRandomGraph(int V, int E, boolean)</code>	Gera grafo aleatório

6.1.2 Geração de Rede Social

Função: `generateSocialNetwork(String namesFile)`

Entrada: Caminho para arquivo de nomes

Saída: Graph (direcionado, 5000 vértices)

1. Carrega 5.000 nomes do arquivo via `NamesLoader`
2. Cria grafo direcionado com 5000 vértices
3. Atribui nomes a cada vértice
4. Para cada pessoa *i*:
 - a. Escolhe aleatoriamente entre 5 e 8 pessoas para seguir
 - b. Arestas têm peso 1 (representa "seguir")
5. Resultado: ~25.000 a 40.000 arestas

6.2 Classe NamesLoader

Utilitário para carregamento do arquivo de nomes.

Método	Descrição
<code>loadNames(String)</code>	Retorna <code>String[]</code> com nomes do arquivo

6.3 Classe PajekIO

Implementa importação e exportação no formato Pajek (.net).

6.3.1 Formato Pajek

```
*Vertices <count>
<id> "<name>"
...
*Arcs
<from> <to> <weight>
...
```

6.3.2 Métodos Estáticos

Método	Descrição
<code>importFrom(String filename)</code>	Lê arquivo em pajek/input/
<code>export(Graph, String filename)</code>	Escreve em pajek/output/

7 Camada de Apresentação

7.1 Classe GraphGUI

Orquestrador principal da interface gráfica, responsável pela estrutura da janela e roteamento de eventos.

7.1.1 Responsabilidades

- Criar estrutura de janela (BorderLayout)
- Inicializar componentes (Header, Sidebar esquerdo, GraphPanel, Sidebar direito)
- Instanciar handlers e injetar dependências
- Roteador de eventos para handlers especializados
- Atualizar UI com resultados

7.1.2 Arquitetura Refatorada

A classe foi refatorada de 1600+ linhas para 250 linhas de orquestração pura:

Responsabilidade Original	Extraída Para
Carregamento/geração de grafos	GraphLoadingHandler
Execução de algoritmos	AlgorithmHandler
Operações de rede social	SocialNetworkHandler
Gerenciamento de estado	GraphUIState
Renderização de grafo	GraphPanel
Design tokens	Theme

7.1.3 Componentes da GUI

GraphGUI (JFrame)

Header: Título, status (V, E, conexo?)

Sidebar Esquerdo

- Gerar
- Importar
- Algoritmos

GraphPanel

(Renderização
do Grafo)

Sidebar Direito

- Estatísticas
- Top Nós
- Resultados

7.2 Classe GraphLoadingHandler

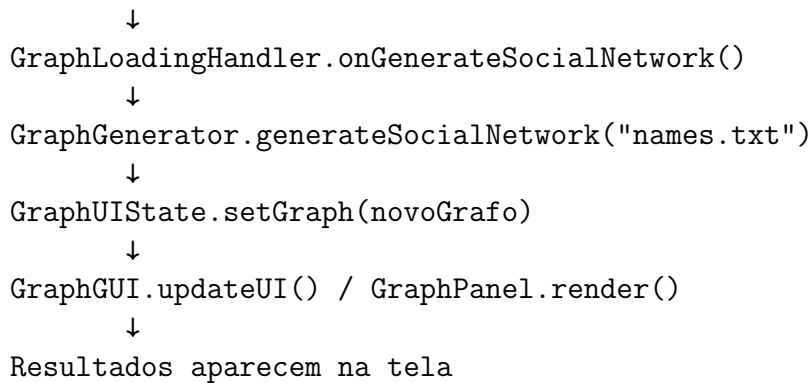
Controller para operações de carregamento e geração de grafos.

7.2.1 Responsabilidades

- Coordenar geração de rede social
- Coordenar geração de grafo aleatório
- Coordenar importação de arquivo Pajek
- Atualizar estado da aplicação
- Chamar callbacks de progresso

7.2.2 Fluxo de Carregamento

Usuário clica "Gerar Rede Social"



7.3 Classe AlgorithmHandler

Controller para execução de algoritmos de análise.

7.3.1 Responsabilidades

- Validar tamanho do grafo
- Executar algoritmos de GraphAlgorithms
- Formatar resultados para exibição
- Alertar usuário sobre operações pesadas
- Atualizar lista de top nós (centralidades)

7.3.2 Métodos Principais

7.4 Classe SocialNetworkHandler

Controller para operações de rede social.

Método	Descrição
<code>checkConnectivity()</code>	Chama <code>algs.isConnected()</code>
<code>getComponents()</code>	Chama <code>algs.getComponents()</code>
<code>checkEulerian()</code>	Chama <code>algs.isEulerian()</code>
<code>hasCycle()</code>	Chama <code>algs.hasCycle()</code>
<code>closenessCentrality()</code>	Chama <code>algs.closenessCentrality()</code>
<code>betweennessCentrality()</code>	Chama <code>algs.betweennessCentrality()</code>

7.4.1 Responsabilidades

- Processar requisições de recomendação
- Validar entrada do usuário
- Chamar `SocialNetwork.recommendFollower()`
- Formatar caminho encontrado para exibição
- Atualizar resultado na interface

7.5 Classe `GraphPanel`

Renderizador visual do grafo em um canvas (`JPanel`).

7.5.1 Responsabilidades

- Desenhar vértices (círculos coloridos)
- Desenhar arestas (setas direcionadas)
- Suportar pan (movimento) interativo
- Suportar zoom (ampliação/redução)
- Usar design tokens de Theme

7.5.2 Algoritmo de Renderização

`paintComponent(Graphics2D g):`

1. Limpa canvas
2. Para cada aresta (u, v):
 - Calcula posição de u e v usando layout (circular ou força)
 - Desenha linha com seta direcionada
3. Para cada vértice u:
 - Desenha círculo colorido
 - Desenha rótulo centralizado
4. Aplica transformações (pan + zoom)

7.6 Classe `GraphUIState`

Gerenciador centralizado de estado da aplicação.

7.6.1 Estado Gerenciado

Propriedade	Tipo
currentGraph	Graph
algorithms	GraphAlgorithms
socialNetwork	SocialNetwork
lastResults	String
topNodes	List<Integer>

7.7 Classe Theme

Sistema de design tokens centralizado.

7.7.1 Tokens Fornecidos

Token	Valor
PRIMARY_COLOR	Cor primária da aplicação
SECONDARY_COLOR	Cor secundária
BACKGROUND_COLOR	Cor de fundo
FONT_FAMILY	Font padrão
FONT_SIZE	Tamanho de fonte padrão
PADDING	Espaçamento padrão

7.8 Componentes Customizados

- **PrimaryButton** — Botão primário com hover effects
- **MenuActionButton** — Botão com ícone e label
- **CollapsiblePanel** — Painel expansível/colapsável
- **ProgressDialog** — Diálogo com barra de progresso
- **VectorIcon** — Ícones vetoriais escaláveis

8 Fluxos de Dados Principais

8.1 Fluxo 1: Inicialização da Aplicação

```
Main.main(String[])
↓
Menu.chooseConsoleMode()
  Se GUI (padrão):
    SwingUtilities.invokeLater()
      new GraphGUI().setVisible(true)
        initComponents()
        createHandlers()
        setupListeners()
        setVisible(true)

  Se Console:
    Main.showMainMenu()
    Loop interativo CLI
```

8.2 Fluxo 2: Geração de Rede Social

```
Usuário clica "Gerar Rede Social"
↓
GraphGUI.onGenerateSocialNetwork() [ActionListener]
↓
GraphLoadingHandler.generateSocialNetwork()
  Mostra ProgressDialog
  GraphGenerator.generateSocialNetwork("names.txt")
    NamesLoader.loadNames("names.txt")
    Graph g = new Graph(5000, true)
    setName(i, nomes[i]) para cada pessoa
    addEdge() para cada relação "segue"
  GraphUIState.setGraph(g)
  Recria GraphAlgorithms e SocialNetwork
  GraphPanel.setGraph(g)
  updateResults()
    Exibe estatísticas no sidebar direito
```

8.3 Fluxo 3: Execução de Algoritmo (Exemplo: Conectividade)

```
Usuário clica "Conectividade"
↓
GraphGUI.onCheckConnectivity() [ActionListener]
↓
AlgorithmHandler.checkConnectivity()
  GraphAlgorithms algs = state.getAlgorithms()
  boolean connected = algs.isConnected()
  Se |V| = 1, retorna true
  BFS no grafo não-direcionado
```

```
Verifica se todos vértices foram visitados
Formata resultado em String
GraphGUI.updateResults(resultado)
Exibe "Connected: Yes/No" no sidebar direito
```

8.4 Fluxo 4: Sistema de Recomendação de Seguidores

```
Usuário digita "Alice" e "Carol"
↓
GraphGUI.onRecommendFollower()
↓
SocialNetworkHandler.recommendFollower("Alice", "Carol")
    SocialNetwork social = state.getSocialNetwork()
    String resultado = social.recommendFollower("Alice", "Carol")
        Busca índices: u = findByName("Alice"), v = findByName("Carol")
        Valida (não nulos, não iguais)
        Verifica se u já segue v diretamente
        Se não, chama dfsPath(u, v, visitados[], caminho[])
            DFS percorre grafo
            Se encontrar v, reconstrói caminho
            Retorna List<Integer> com sequência de vértices
        Formata resultado: "Recommended! Path: Alice → ... → Carol"
    GraphGUI.updateResults(resultado)
    GraphPanel destaca caminho (opcional)
```

8.5 Fluxo 5: Centralidade de Intermediação (Brandes)

```
Usuário clica "Betweenness Centrality"
↓
AlgorithmHandler.betweennessCentrality()
    Alerta: "Large graph (5000 vertices). Continue? (y/n)"
    GraphAlgorithms algs = state.getAlgorithms()
    double[] b = algs.betweennessCentrality()
        Para cada vértice s (0 a 4999):
            dijkstra(s) → calcula distâncias de s
            Reconstrói árvore de caminhos mínimos
            Acumula dependências em betweenness[]
    Ordena vértices por betweenness decrescente
    Extrai top 10
    GraphGUI.updateResults() e lstTopNodes
        Exibe nomes e valores no sidebar direito
```

9 Padrões de Design

9.1 MVC (Model-View-Controller)

Componente	Papel	Classe(s)
Model	Dados e lógica	Graph, GraphAlgorithms, SocialNetwork
View	Apresentação	GraphGUI, GraphPanel, componentes customizados
Controller	Orquestração	Handlers (Loading, Algorithm, SocialNetwork)

9.2 Facade

GraphGUI expõe interface unificada para manipulação de handlers:

Listing 3: Facade em GraphGUI

```

1 public void onGenerateSocialNetwork() {
2     loadingHandler.generateSocialNetwork();
3 }
4
5 public void onCheckConnectivity() {
6     algorithmHandler.checkConnectivity();
7 }
8
9 public void onRecommendFollower(String src, String tgt) {
10     socialHandler.recommendFollower(src, tgt);
11 }

```

9.3 Dependency Injection

Handlers recebem dependências via construtor:

Listing 4: Injeção de Dependência

```

1 // Em GraphGUI.initComponents():
2 GraphUIState state = new GraphUIState();
3 GraphLoadingHandler handler =
4     new GraphLoadingHandler(state, graphPanel, progressCallback);

```

9.4 Factory

GraphGenerator usa factory methods estáticos:

Listing 5: Factory Pattern

```

1 Graph g1 = GraphGenerator.generateSocialNetwork("names.txt");
2 Graph g2 = GraphGenerator.generateRandomGraph(100, 500, true);

```

9.5 Strategy

Múltiplos algoritmos em GraphAlgorithms:

Listing 6: Strategy Pattern

```
1 double[] closeness = algs.closenessCentrality(); // Dijkstra
2 double[] between = algs.betweennessCentrality(); // Brandes
3 boolean hasCycle = algs.hasCycle(); // DFS
   Color a = o
```

9.6 Observer

Callbacks para atualização de UI:

Listing 7: Observer Pattern

```
1 loadingHandler.setProgressCallback((percent) -> {
2     progressDialog.setValue(percent);
3     SwingUtilities.invokeLater(() -> repaint());
4 });
```

10 Análise de Dependências

10.1 Matriz de Dependências entre Classes

Classe	Depende De	Usado Por	Tipo
Graph	Nenhuma	Todos	Estrutura
GraphAlgorithms	Graph	Handlers, Main	Algoritmos
SocialNetwork	Graph	Handlers, Main	Domínio
GraphGenerator	Graph, NamesLoader	Main, Handlers	Factory
NamesLoader	File I/O	GraphGenerator	Utilitário
PajekIO	Graph, File I/O	Main, Handlers	I/O
GraphGUI	Todos os Handlers	Main	Orquestrador
GraphLoadingHandler	GraphUIState, GraphPanel	GraphGUI	Controller
AlgorithmHandler	GraphUIState	GraphGUI	Controller
SocialNetworkHandler	GraphUIState	GraphGUI	Controller
GraphUIState	Graph, Algs, Social	Handlers	Estado
GraphPanel	Graph	GraphGUI, Handlers	Vista
Theme	Nenhuma	Componentes	Tokens

10.2 Direção das Dependências

GUI (não depende de backend)

↓

Handlers (orquestram operações)

↓

GraphUIState (gerencia estado)

↓

Graph ← GraphAlgorithms (não-direcionado, usa o mesmo Graph)
 ← SocialNetwork (não-direcionado, usa o mesmo Graph)

↓

GraphGenerator, PajekIO, NamesLoader (utilitários para criar/carregar)

10.3 Garantias de Acoplamento Baixo

- **Algoritmos são puros:** GraphAlgorithms não conhece GUI
- **Domínio é puro:** SocialNetwork não conhece Handlers
- **Graph é read-only:** Não há callbacks ou listeners
- **Handlers recebem dependências:** Não instanciam suas dependências
- **Inversão de controle:** Main/GraphGUI orquestram, não as classes

11 Análise de Complexidade

11.1 Algoritmos

Algoritmo	Tempo	Espaço	Observação
Conectividade	$O(V+E)$	$O(V)$	BFS
Componentes	$O(V+E)$	$O(V)$	BFS
Euleriano	$O(V+E)$	$O(V)$	Contagem de graus
Ciclo	$O(V+E)$	$O(V)$	DFS com coloração
Dijkstra	$O((V+E)\log V)$	$O(V)$	Heap binário
Closeness	$O(V(V+E)\log V)$	$O(V)$	$V \times \text{Dijkstra}$
Betweenness (Brandes)	$O(V(V+E)\log V)$	$O(V)$	$V \times \text{Dijkstra}$

11.2 Complexidade para Rede Social ($V=5000$, $E=30000$)

Operação	Complexidade Teórica	Tempo Estimado
Conectividade	$O(35000)$	~1 segundo
Componentes	$O(35000)$	~1 segundo
Euleriano	$O(35000)$	~1 segundo
Ciclo	$O(35000)$	~1 segundo
Closeness (Top 10)	$O(5000 \times 35000 \times \log 5000)$	5-10 minutos
Betweenness (Top 10)	$O(5000 \times 35000 \times \log 5000)$	5-10 minutos
Recomendação (DFS)	$O(35000)$ no pior caso	~1 segundo típico

Nota: Centralidades são operações pesadas. A aplicação alerta usuário antes de executar em grafos > 500 vértices.

12 Estrutura de Pacotes e Responsabilidades

12.1 Pacote `graph.model`

Classe	Responsabilidade
Graph	Estrutura de dados de grafo

12.2 Pacote `graph.algorithm`

Classe	Responsabilidade
GraphAlgorithms	7 algoritmos + helpers privados

12.3 Pacote `graph.domain`

Classe	Responsabilidade
SocialNetwork	Lógica de recomendação + DFS

12.4 Pacote `graph.io`

12.5 Pacote `graph.generator`

12.6 Pacote `graph.gui`

12.7 Pacote `graph` (raiz)

Classe	Responsabilidade
PajekIO	Importação/exportação (estático)
NamesLoader	Carregamento de nomes (estático)

Classe	Responsabilidade
GraphGenerator	Factory para criação de grafos (estático)

13 Princípios SOLID Aplicados

13.1 S — Single Responsibility Principle

Cada classe tem uma única responsabilidade bem definida:

- **Graph** — Apenas armazenar e consultar estrutura
- **GraphAlgorithms** — Apenas executar algoritmos
- **SocialNetwork** — Apenas lógica de domínio
- **GraphGUI** — Apenas orquestração de UI
- **Handlers** — Apenas roteamento de eventos

13.2 O — Open/Closed Principle

Estrutura aberta para extensão, fechada para modificação:

- Novos algoritmos podem ser adicionados a **GraphAlgorithms** sem modificar código existente
- Novos componentes customizados podem ser criados herdando de **JComponent**
- Novo handlers podem ser criados herdando interface padrão

13.3 L — Liskov Substitution Principle

Componentes customizados substituem **JComponent** sem quebrar contrato:

Listing 8: LSP em Componentes Customizados

```

1 JComponent btn = new PrimaryButton("Click me");
2 JComponent panel = new CollapsiblePanel("Title");
3 // Ambos podem ser usados onde JComponent esperado

```

13.4 I — Interface Segregation Principle

Interfaces pequenas e focadas:

Listing 9: ISP em Listeners

```

1 loadingHandler.setProgressCallback(value -> {...});
2 // N o f o r a m o s h a n d l e r a i m p l e m e n t a r i n t e r f a c e m o n o l i t i c a

```

Classe	Responsabilidade
GraphGUI	Orquestração de UI + roteamento de eventos
GraphPanel	Renderização de grafo
GraphUIState	Gerenciamento centralizado de estado
GraphLoadingHandler	Carregamento/geração
AlgorithmHandler	Execução de algoritmos
SocialNetworkHandler	Recomendações
ProgressDialog	Diálogo de progresso
Theme	Design tokens
PrimaryButton	Botão customizado
MenuItemButton	Botão de ação customizado
CollapsiblePanel	Painel expansível
VectorIcon	Ícones vetoriais

Classe	Responsabilidade
Main	Ponto de entrada + menu CLI
Menu	Seletor GUI/CLI

13.5 D — Dependency Inversion Principle

Dependências apontam para abstrações, não para concretos:

Listing 10: DIP em Handlers

```

1 // Handler n o cria GraphUIState, recebe via construtor
2 handler = new AlgorithmHandler(state);
3 // Desacopla cria o de inje o
```

14 Refatorizações Realizadas

14.1 Refatoração 1: Extração de Algoritmos

Antes: Graph era uma God Class com 340 linhas contendo algoritmos, geração e I/O.

Depois:

- Algoritmos extraídos para `GraphAlgorithms` (classe dedicada)
- Geração extraída para `GraphGenerator`
- I/O extraído para `PajekIO`
- Graph reduzida para 100 linhas de estrutura pura

14.2 Refatoração 2: Refactoring de GUI

Antes: GraphGUI era monolítica com 1600+ linhas.

Depois:

- Orquestração pura em `GraphGUI` (250 linhas)
- Carregamento em `GraphLoadingHandler`
- Algoritmos em `AlgorithmHandler`
- Domínio em `SocialNetworkHandler`
- Estado em `GraphUIState`
- Renderização em `GraphPanel`
- Design tokens em `Theme`

14.3 Refatoração 3: Encapsulamento

Antes: Campo `adj` era package-private, permitindo acesso direto.

Depois:

- `adj` agora é `private`
- Novo método público `neighbors(int)` para acesso seguro
- `GraphAlgorithms` e `SocialNetwork` usam `neighbors()` em vez de acesso direto

14.4 Refatoração 4: Eliminação de Duplicação

Antes: Código de geração de nomes era duplicado.

Depois:

- Classe dedicada `NamesLoader`
- `GraphGenerator` chama `NamesLoader`
- Uma única fonte de verdade para carregamento de nomes

15 Conclusão

15.1 Arquitetura Alcançada

GraphNet Analyzer demonstra uma arquitetura bem estruturada, seguindo os princípios SOLID e padrões de design clássicos. A separação clara entre camadas (Apresentação, Domínio, Algoritmos, Estrutura de Dados) resulta em código:

- **Modular:** Cada classe tem responsabilidade bem definida
- **Testável:** Componentes podem ser testados isoladamente
- **Extensível:** Novos algoritmos/componentes podem ser adicionados
- **Manutenível:** Baixo acoplamento facilita manutenção
- **Reusável:** Classes podem ser usadas em outros projetos

15.2 Características Principais

1. **Estrutura de Dados Eficiente:** Lista de adjacências para $O(1)$ de adição de arestas
2. **7 Algoritmos Clássicos:** Cobertura completa de análise de grafos
3. **Interface Gráfica Moderna:** Visualização interativa com componentes customizados
4. **Sistema de Recomendação:** DFS com rastreamento de caminho completo
5. **Múltiplos Modos:** GUI e CLI para diferentes casos de uso
6. **I/O Flexível:** Formato Pajek para interoperabilidade

15.3 Autoria

Este projeto foi desenvolvido colaborativamente por:

- **Jafte Carneiro Fagundes da Silva** (@cyberfika)
- **Nicolas Hrescak** (@NicolasHrescak)

Sob orientação do Prof. Fabrício Enembreck, PUCPR — Pontifícia Universidade Católica do Paraná.

Todos os arquivos de código foram criados pelos autores, sem uso de templates ou geradores de código automático. A documentação interna (Javadoc) está em 100% de cobertura em português.

15.4 Recomendações para Expansão Futura

1. **Layout de Visualização:** Implementar Force-Directed Layout (Fruchterman-Reingold) para melhor visualização de grafos grandes
2. **Persistência:** Adicionar suporte a banco de dados para grafos muito grandes (>50k vértices)
3. **Testes Automatizados:** Implementar suite de testes JUnit para aumentar confiabilidade
4. **Paralelização:** Usar ExecutorService para paralelizar Dijkstra/Brandes
5. **APIs Adicionais:** Expor como REST API para consumo por outras aplicações

A Referências de Código

A.1 Arquivo de Nomes

Localização: data/names.txt

Formato: Um nome brasileiro por linha (5.000 nomes)

A.2 Formato Pajek

Arquivo de exemplo: pajek/input/example.net

```
*Vertices 3
1 "Alice"
2 "Bob"
3 "Carol"
*Arcs
1 2 1
2 3 1
1 3 1
```

A.3 Comando de Compilação

```
javac -encoding UTF-8 -d out \
  src/graph/model/Graph.java \
  src/graph/algorithm/GraphAlgorithms.java \
  src/graph/domain/SocialNetwork.java \
  src/graph/io/PajekIO.java \
  src/graph/io/NamesLoader.java \
  src/graph/generator/GraphGenerator.java \
  src/graph/gui/Theme.java \
  src/graph/gui/VectorIcon.java \
  src/graph/gui/GraphUIState.java \
  src/graph/gui/ProgressDialog.java \
  src/graph/gui/PrimaryButton.java \
  src/graph/gui/MenuActionButton.java \
```

```
src/graph/gui/CollapsiblePanel.java \  
src/graph/gui/GraphPanel.java \  
src/graph/gui/GraphLoadingHandler.java \  
src/graph/gui/AlgorithmHandler.java \  
src/graph/gui/SocialNetworkHandler.java \  
src/graph/gui/GraphGUI.java \  
src/graph/Menu.java \  
src/graph/Main.java
```

A.4 Comando de Execução

```
java -cp out graph.Main
```

Ao iniciar, escolha:

- 1 Interface Gráfica (padrão, recomendado)
- 2 Modo Console (legado)